



MULTIDISCIPLINARY PROJECT (CO3109)

Assignment

“Smart Agriculture System with Edge Computing and Data Analysis”

Instructor(s): Diệp Thanh Đăng

Students: Nguyễn Xuân Huy Hoàng - 2311070 (Leader)

Vũ Trọng Nghĩa - 2312291

Nguyễn Trần Yến Nhi - 2312506

Bùi Trọng Hiên - 2310993

Ngô Quang Hiệu - 2311015

Nguyễn Lưu Khánh Trình - 2313638



Contents

Member list & Workload	3
List of Figures	3
List of Tables	3
1 Introduction	4
1.1 Project objectives	4
1.2 Project scope	4
1.3 Stakeholders	4
2 System Architecture Overview	5
2.1 Overall Model	5
2.2 Middleware Platform	5
2.3 Routing and Load Balancing Mechanism for HTTP Requests	5
3 Requirements analysis	6
3.1 Definitions and Terminology	6
3.2 Functional requirements	6
3.3 Non-functional requirements	7
4 Use-case design	8
4.1 Use-case diagram	8
4.2 Use-case details	8
5 Software design	13
5.1 Class diagram	13
5.2 Applied design patterns	13
5.2.1 Observer pattern	13
5.2.2 Strategy pattern	14
5.2.3 Singleton pattern	14
5.3 Database design	14
5.3.1 Time-Series Database (e.g., InfluxDB) for Telemetry	15
5.3.2 Relational Database (e.g., PostgreSQL/MySQL) for State and Logic	16
5.4 API and Communication Design	16
5.4.1 MQTT Communication (Edge to Cloud)	17
5.4.2 RESTful APIs (Frontend to Backend)	17
6 Implementation and Integration	19
6.1 Edge Hardware Setup	19
6.1.1 Overview of the Role of the Edge Device in the System	19
6.1.2 Hardware Architecture	19
6.1.3 Connection Process for Data Transmission from Edge to Cloud	21
6.1.4 Cloud Connection Configuration (Adafruit IO Integration)	21
6.1.5 Experimentation and Evaluation	21
6.2 Load Balancer Configuration	21
6.3 Backend API Development and Web Interface Integration	22
6.4 Data Analysis Module	22
7 Conclusion and Future Work	25
7.1 Evaluation of Achieved Results	25
7.2 Future Development and System Expansion	25

List of Figures

1	Use-case Diagram	8
2	Class Diagram	13
3	Entity-Relationship Diagram (Relational Schema) and Time-Series Data Flow	15
4	Yolo:Bit Extend Shield Illustration	19
5	Peripheral sensors and indicator modules	20
6	Experimental results	21
7	Data Refresh Mechanism	23
8	Dashboard Overview	23
9	Time Filtering and Date Range Selection	24
10	Status Evaluation and Alert	24
11	Recommendation Interface	24

List of Tables

1	Member list & workload	3
2	Project stakeholders	4
3	Use Case: Configure ideal range	9
4	Use case: Configure critical range	10
5	Use Case: Control actuators (manual)	11
6	Use Case: Monitor environmental data	12
7	Use Case: View environmental data history	12
8	Core RESTful API Endpoints	17



Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Nguyễn Xuân Huy Hoàng	2311070	- ok - ok	100%
2	Vũ Trọng Nghĩa	2312291	- ok - ok	100%
5	Nguyễn Trần Yến Nhi	2312506	- ok - ok	100%
3	Bùi Trọng Hiền	2310993	- ok - ok	100%
4	Ngô Quang Hiệu	2311015	- Data analyst	100%
5	Nguyễn Lưu Khánh Trình	2313638	- ok - ok	100%

Table 1: Member list & workload

1 Introduction

1.1 Project objectives

This project aims to implement a smart agriculture model that leverages modern technologies to address challenges posed by climate change while optimizing labor and resource usage. The key objectives of the project include:

- Collecting real-time data on temperature, air humidity, and light intensity to maintain suitable environmental conditions for plant growth.
- Automatically activating irrigation systems and ventilation mechanisms based on predefined thresholds, helping to conserve resources and stabilize environmental conditions.
- Providing an intuitive user interface that allows users to monitor and control the system remotely via web applications.
- Applying software engineering principles such as Design Patterns, Load Balancing, and Containerization (Docker) to build a scalable and resilient system.

1.2 Project scope

This project focuses on designing and implementing a small-scale smart greenhouse system that incorporates the following core technical components:

- Developing a hardware infrastructure based on the Yolo:Bit microcontroller, integrated with environmental sensors including the DHT20 (temperature and air humidity), a light sensor (LDR), and an infrared sensor for motion detection. The system also incorporates devices such as an LCD/OLED display, RGB LED, fan, and servo motor.
- Establishing an automatic operation mechanism (Auto Mode), allowing the system to respond to environmental conditions (e.g., temperature or light intensity) based on user-defined thresholds.
- Developing a dashboard to support manual interaction (Manual Mode), enabling users to monitor data and directly control devices when necessary.
- Implementing a cloud-connected system where the edge device transmits data via MQTT (Adafruit IO), enabling real-time monitoring and remote control.

The project focuses on data processing and control logic within an IoT system; advanced features such as soil moisture monitoring or AI-based analysis are not included in the scope.

1.3 Stakeholders

Stakeholder	Role	Expectation
Farm Owner	Primary user	Easy-to-use interface, reliable system operation, and reduced risk of crop damage due to unfavorable environmental conditions.
Technician	System maintenance & Administrator	Easy system configuration, detailed logs for monitoring, and efficient troubleshooting of hardware and system issues.

Table 2: Project stakeholders

2 System Architecture Overview

2.1 Overall Model

The system follows a hierarchical IoT architecture that facilitates seamless two-way communication between physical sensors and the user interface.

- **Uplink (Data Flow):** Environmental data is captured by Edge Devices (Yolo:Bit/ESP32) → Transmitted via MQTT to the Adafruit IO Broker → Retrieved by the Load Balancer's Subscriber → Routed to an available Backend Server → Stored and processed → Visualized on the Frontend Application.
- **Downlink (Control Flow):** Control commands (Manual or Automated) originate in the Backend → Dispatched to Adafruit IO via MQTT Publish → Received by the Edge Gateway → Executed by Actuators (Pumps, Fans, LEDs).

2.2 Middleware Platform

The architecture utilizes **Adafruit IO** as the core Middleware Platform, serving as a reliable MQTT Broker.

- **MQTT Protocol:** Chosen for its lightweight overhead and low power consumption, MQTT is ideal for the Edge layer where bandwidth and energy resources are constrained. The Publish/Subscribe model allows for asynchronous communication, ensuring that Edge devices do not need to wait for a synchronous HTTP response.
- **Feed Management:** Adafruit IO organizes data into "Feeds," allowing the system to scale by simply adding new topics for additional sensor nodes or actuators.

2.3 Routing and Load Balancing Mechanism for HTTP Requests

To maintain a high system uptime ($\geq 99\%$) and handle up to 50 concurrent sensor nodes, a custom Load Balancing mechanism is employed.

- **Load Distribution:** The Load Balancer intercepts all incoming HTTP traffic. Using a **Round-Robin** algorithm, it maintains an index of healthy backend instances and cycles through them for each new request.
- **Scalability and Performance:** This approach prevents any single backend instance from becoming a bottleneck. It allows the system to handle bursts of telemetry data and high volumes of concurrent user requests by distributing the computational load, thereby ensuring low-latency responses across the entire ecosystem.

3 Requirements analysis

3.1 Definitions and Terminology

The following terms are used consistently throughout this section:

- **Environmental metrics:** The physical parameters monitored by the system, including temperature, air humidity, and light intensity.
- **Ideal range:** The preferred range of environmental values that ensures optimal plant growth.
- **Critical range:** The acceptable safety range of environmental values. Values outside this range are considered dangerous.
- **Actuator:** A device that performs actions based on system commands (e.g., fan, servo motor).
- **Auto Mode:** A system mode in which actuators are controlled automatically based on predefined parameters and environmental conditions.
- **Manual Mode:** A system mode in which users directly control actuators.

3.2 Functional requirements

User (Farm Owner)

- The user can log into the system using authorized credentials to access the smart greenhouse management dashboard.
- The user can configure the ideal range for environmental metrics, allowing the system to automatically control actuators in Auto Mode.
- The user can define the critical range for environmental metrics to trigger alerts on the RGB LED and dashboard, and to activate system responses.
- The user can monitor real-time environmental data via the on-site LCD display or through the dashboard.
- The user can manually turn actuators on or off using the Manual Mode on the dashboard.
- The user can view alert history, including timestamps and alert types (e.g., environmental, system or security alerts), through the dashboard.
- The user can view historical charts of the environmental metrics.
- The user can review the activity history of actuators, including on/off timestamps, trigger sources (automatic or manual), and statistics on operation duration within a specific period.

Non-interactive functional requirements

- The system sends confirmation notifications via the dashboard when an actuator is successfully turned on or off by the automatic control logic.
- The system generates alerts via the dashboard when an actuator fails to change its state after a control command has been issued.
- The system sends notifications via the dashboard when a sensor node or the Yolo:Bit microcontroller loses Wi-Fi/Internet connectivity beyond a predefined duration.
- The system generates alerts via the dashboard when sensors report abnormal values (e.g., humidity = 0% or temperature $> 100^{\circ}C$), indicating potential hardware malfunction.
- The system sends warning notifications via the dashboard when environmental metrics begin to deviate from the ideal range but have not yet exceeded the critical range.
- The system triggers emergency alerts via the dashboard and on-site RGB LED, and displays warning messages on the LCD, when environmental metrics exceed the critical range, requiring immediate user intervention.

- The system generates an intrusion alert when the infrared sensor detects motion, displaying the warning on the LCD, sending a notification via the dashboard, and recording the event for later review.

3.3 Non-functional requirements

Performance

- Manual control commands issued by the user must be executed by the actuators within 1 second.
- Alert notifications must be delivered within 5 seconds from the time an incident is detected.
- The data sampling interval from sensors must not exceed 10 seconds.

Scalability

- The system must be capable of handling concurrent data from at least 50 sensor nodes without performance degradation or bottlenecks.

Security

- Only authorized users are allowed to modify the ideal range and the critical range, and to control actuators.
- All data transmitted between the Edge devices and the Server must be encrypted to prevent unauthorized access, tampering, or spoofing of environmental data.

Availability & Reliability

- The uptime of core server services (including the backend API, database, and load balancer) must be at least 99%.

Usability & Maintainability

- Basic operations such as configuring thresholds, controlling actuators, and viewing historical data must be completed within a maximum of three steps.
- The system must provide comprehensive documentation for installation, operation, and maintenance, enabling users to handle basic issues without technical support.
- The backend services and system components must be containerized using Docker to ensure consistent deployment and ease of maintenance.
- Historical data of environmental metrics and device activities must be stored for at least one year to support trend analysis and system optimization.

4 Use-case design

4.1 Use-case diagram

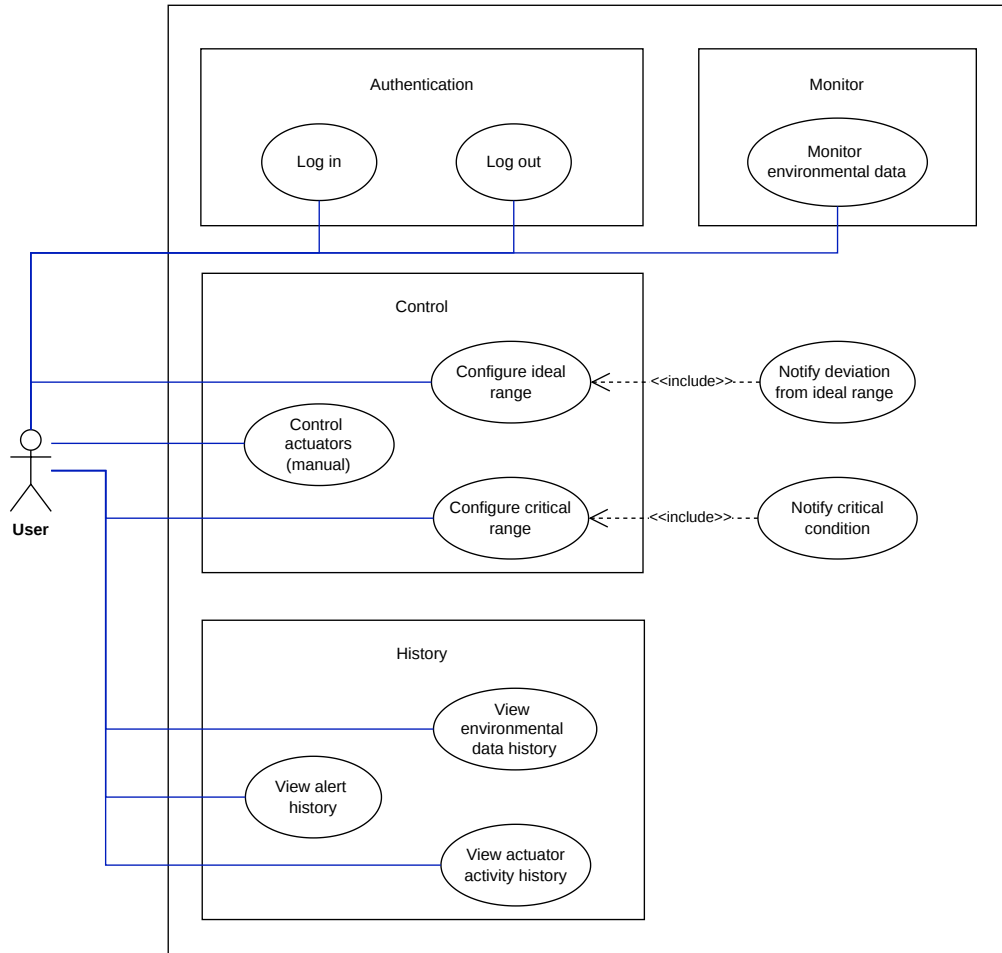


Figure 1: Use-case Diagram

4.2 Use-case details

Use Case	Configure ideal range
Actors	User
Description	The user defines the ideal range for environmental metrics to enable the system to automatically control actuators in Auto Mode.
Preconditions	• The user has successfully logged into the system. • The system is operational and maintains a stable connection with sensor nodes.

Postconditions	• The newly configured ideal ranges are immediately applied to the system's automatic control logic in Auto Mode. • The configuration changes are recorded in the system log for auditing and management purposes.
Trigger	The user selects the "Ideal range" option from the dashboard menu.
Normal Flow	1. The system displays the current environmental metrics along with input fields for defining ideal ranges. 2. The user enters desired value ranges (e.g., temperature $25^{\circ}C - 28^{\circ}C$, humidity $60\% - 70\%$). 3. The user clicks the "Save" button. 4. The system validates the input data (e.g., minimum value must be less than maximum value, correct data format). 5. The system stores the new ideal ranges in the database and sends updated parameters to the edge device. The system then displays a confirmation message: "Configuration updated successfully".
Alternative Flows	3a. The user selects "Cancel"; the system discards all changes and retains the previous configuration.
Exceptions	5a. If the user enters invalid data (e.g., non-numeric values or inconsistent ranges), the system displays an error message and requests correction.

Table 3: Use Case: Configure ideal range

Use Case	Configure critical range
Actors	User
Description	The user defines the critical range for environmental metrics. When real-time values reach or exceed these thresholds, the system overrides normal control logic, activates emergency responses (e.g., running actuators at maximum capacity), and issues alerts.
Preconditions	• The user has successfully logged into the system. • The system is operational and maintains a stable connection with sensor nodes.
Postconditions	• The newly configured critical ranges are applied to the system's monitoring and emergency handling logic. • The configuration is stored in the database and recorded in the system log.
Trigger	The user selects the "Critical range" option on the dashboard.

Normal Flow	1. The system displays current critical thresholds for environmental metrics. 2. The user inputs new threshold values. 3. The user clicks "Confirm". 4. The system validates the input (e.g., critical thresholds must not conflict with the ideal range). 5. The system stores the new configuration in the database and updates the system logic. 6. The system displays a confirmation message: "Critical range configuration updated successfully".
Alternative Flows	3a. The user cancels the operation; the system discards all changes and retains the previous configuration.
Exceptions	4a. If the input values are logically inconsistent (e.g., critical thresholds overlap or fall within the ideal range), the system rejects the input and requests correction.

Table 4: Use case: Configure critical range

Use Case	Control actuators (manual)
Actors	User
Description	The user manually turns actuators on or off via the Dashboard. When manual control is activated, the system temporarily prioritizes user commands over automatic control based on environmental conditions and predefined ideal ranges.
Preconditions	• The user has successfully logged into the system. • The system is operational and connected to edge devices and sensors. • Actuators are available and responsive.
Postconditions	• The physical state of the selected actuator is updated according to the user's command. • The system switches the selected actuator to Manual Mode until it is reset by the user.
Trigger	The user selects an actuator and activates a control action (ON/OFF) from the manual control panel on the Dashboard.

Normal Flow	1. The user selects an actuator and presses the ON/OFF control button. 2. The system validates the user's permission and receives the control request. 3. The system sends a control signal to the edge device (via relay control mechanism). 4. The edge device executes the command by switching the corresponding relay state. 5. The system updates the actuator status on the Dashboard in real time. 6. The system logs the action to database, including username, time, actuator name, and status. 7. The selected actuator is set to Manual Mode, temporarily overriding Auto Mode control logic.
Alternative Flows	1a. The user sets a timed manual control (e.g., turn on fan for 5 minutes). After the duration expires, the system automatically turns the actuator off and returns control to Auto Mode.
Exceptions	3a. If the system does not receive confirmation from the edge device or relay state does not change, the system raises an alert: "Device error: [Actuator] is not responding".

Table 5: Use Case: Control actuators (manual)

Use Case	Monitor environmental data
Actors	User
Description	The user monitors real-time environmental metrics including temperature, air humidity, and light intensity through the Dashboard. The system also provides local visual feedback on the edge device (LCD screen and RGB LED), which acts as an auxiliary display for real-time on-site observation.
Preconditions	• The user has successfully logged into the system. • The system is operational and connected to sensor nodes.
Postconditions	• The user obtains up-to-date environmental information for decision-making or monitoring purposes.
Trigger	The user accesses the Dashboard interface.
Normal Flow	1. The user opens the Dashboard page. 2. The system displays real-time environmental metrics (temperature, air humidity, light intensity). 3. The user observes the data and interprets the current environmental conditions. 4. The system visually highlights abnormal values if any metric exceeds predefined thresholds (ideal or critical ranges).

Alternative Flows	1a. The user monitors environmental data directly from the edge device LCD screen instead of the Dashboard. 1b. The user observes RGB LED status indicators as a quick visual summary of system state.
Exceptions	1a. If sensor data is unavailable or invalid, the system displays "N/A" or "Error" on the Dashboard. 1b. If data transmission fails, the Dashboard shows the last updated values while the system continues attempting to reconnect.

Table 6: Use Case: Monitor environmental data

Use Case	View environmental data history
Actors	User
Description	The user views and analyzes historical environmental data (temperature, air humidity, and light intensity) through the Dashboard. The system provides interactive analytics including filtering, aggregation, visualization, recommendations, and data export to support decision-making.
Preconditions	• The user has successfully logged into the system. • Historical environmental data is available in the system.
Postconditions	• The user successfully views historical environmental data and analytics on the Dashboard.
Trigger	The user navigates to the historical data analysis section of the Dashboard.
Normal Flow	1. The system displays environmental data visualizations for temperature, humidity, and light intensity. 2. The user performs analysis actions such as: - selecting time filters or custom date ranges - changing aggregation level (raw, daily, weekly, monthly) - viewing system-generated recommendations - exporting data in CSV format 3. The system updates the displayed analytics accordingly.
Alternative Flows	
Exceptions	1a. If data retrieval fails, the system displays cached data with a warning indicator. 3a. If data is unavailable for the selected range, the system displays an empty state message. 1c. If export fails, the system notifies the user and allows retry.

Table 7: Use Case: View environmental data history

5 Software design

5.1 Class diagram

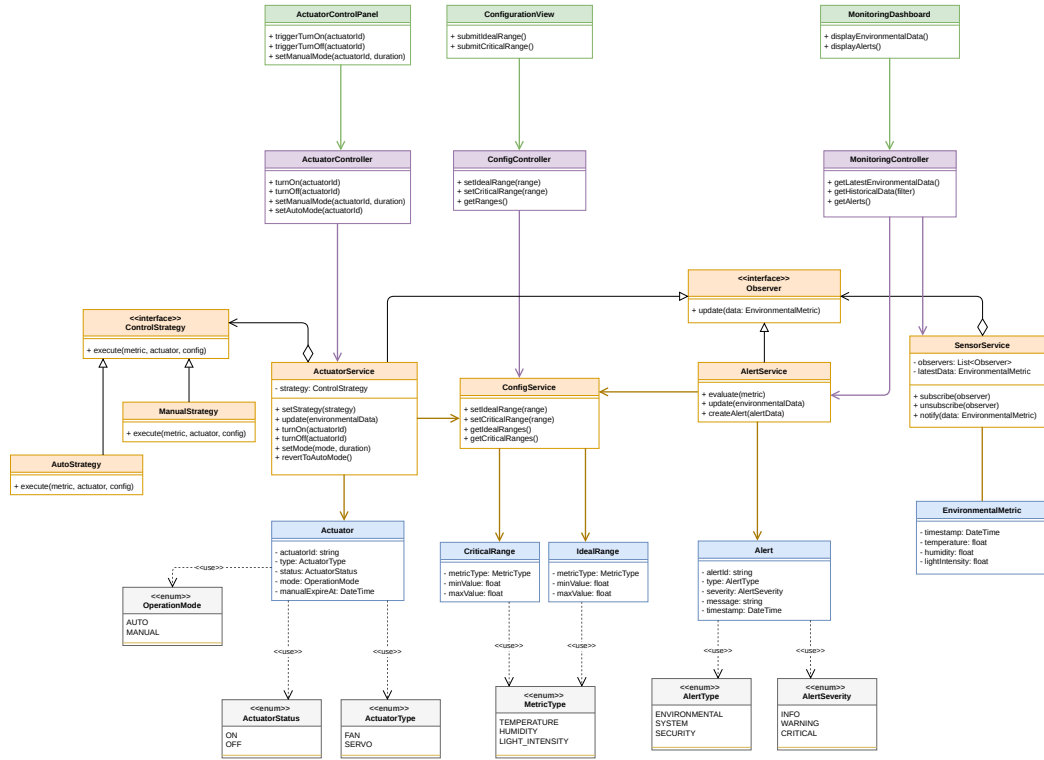


Figure 2: Class Diagram

The class diagram focuses on modeling the core control logic of the system while emphasizing the application of key design patterns, including Observer and Strategy, within an MVC-inspired architecture.

Specifically, the diagram includes the main components required for real-time environmental monitoring and actuator control, namely controllers, services, entities, and views. These components collaborate to support essential functionalities such as receiving sensor data, evaluating environmental conditions, and controlling actuators in both automatic and manual modes.

To maintain clarity and avoid unnecessary complexity, the diagram intentionally excludes secondary features such as historical data management, data analysis, and recommendation generation. These aspects are considered supporting functionalities and are handled at the persistence or application layer rather than being explicitly modeled in the class diagram.

This design decision ensures that the diagram remains focused on the system's primary control flow and the interaction between components that implement the core behavior.

5.2 Applied design patterns

5.2.1 Observer pattern

The Observer pattern is used to enable real-time data propagation between system components in a loosely coupled manner. In this pattern, a subject maintains a list of observers and notifies them automatically whenever its state changes.

In the proposed system, the **SensorService** acts as the subject, while components such as **AlertService** and **ActuatorService** act as observers. When new environmental data is received from sensor nodes,

the **SensorService** updates its internal state and notifies all subscribed observers through the `update()` method.

This design allows multiple services to react to the same data stream independently. For example, the **AlertService** evaluates the data to generate alerts, while the **ActuatorService** uses it to determine control actions based on the current operating strategy.

By applying the Observer pattern, the system achieves better scalability and flexibility, as new observers can be added without modifying the **SensorService**. It also improves maintainability by reducing tight coupling between data producers and data consumers.

5.2.2 Strategy pattern

The Strategy pattern is applied to encapsulate different control behaviors of actuators and allow dynamic switching between them at runtime.

In this system, the control logic is divided into two main strategies: **AutoStrategy** and **ManualStrategy**, both implementing the common interface **ControlStrategy**. The **ActuatorService** acts as the context, holding a reference to the current strategy and delegating the decision-making process to it.

In Auto mode, the **AutoStrategy** evaluates environmental data against configured ideal and critical ranges to determine appropriate control actions (e.g., turning actuators on or off). In contrast, the Manual mode allows user-defined control, temporarily overriding automatic behavior for a specified duration.

By using the Strategy pattern, the system can switch between different control modes without modifying the core logic of the **ActuatorService**. This improves flexibility, supports extensibility for future control strategies, and ensures a clear separation of concerns between control policies and execution logic.

5.2.3 Singleton pattern

The Singleton pattern is applied to ensure that certain system components have a single, shared instance throughout the application lifecycle.

In this system, both the database connection and the MQTT client connection are implemented as singletons. This design prevents the overhead and inconsistency that may arise from creating multiple connections, while ensuring efficient resource usage and centralized management.

These components are treated as infrastructure-level concerns and are therefore not explicitly represented in the class diagram.

5.3 Database design

To address the specific demands of an IoT-based smart agriculture system—characterized by high-frequency sensor data ingestion and complex state management the system adopts a **Hybrid Database Architecture** (Polyglot Persistence). This design utilizes a Time-Series Database (TSDB) alongside a traditional Relational Database Management System (RDBMS) to optimize both read/write performance and data integrity.

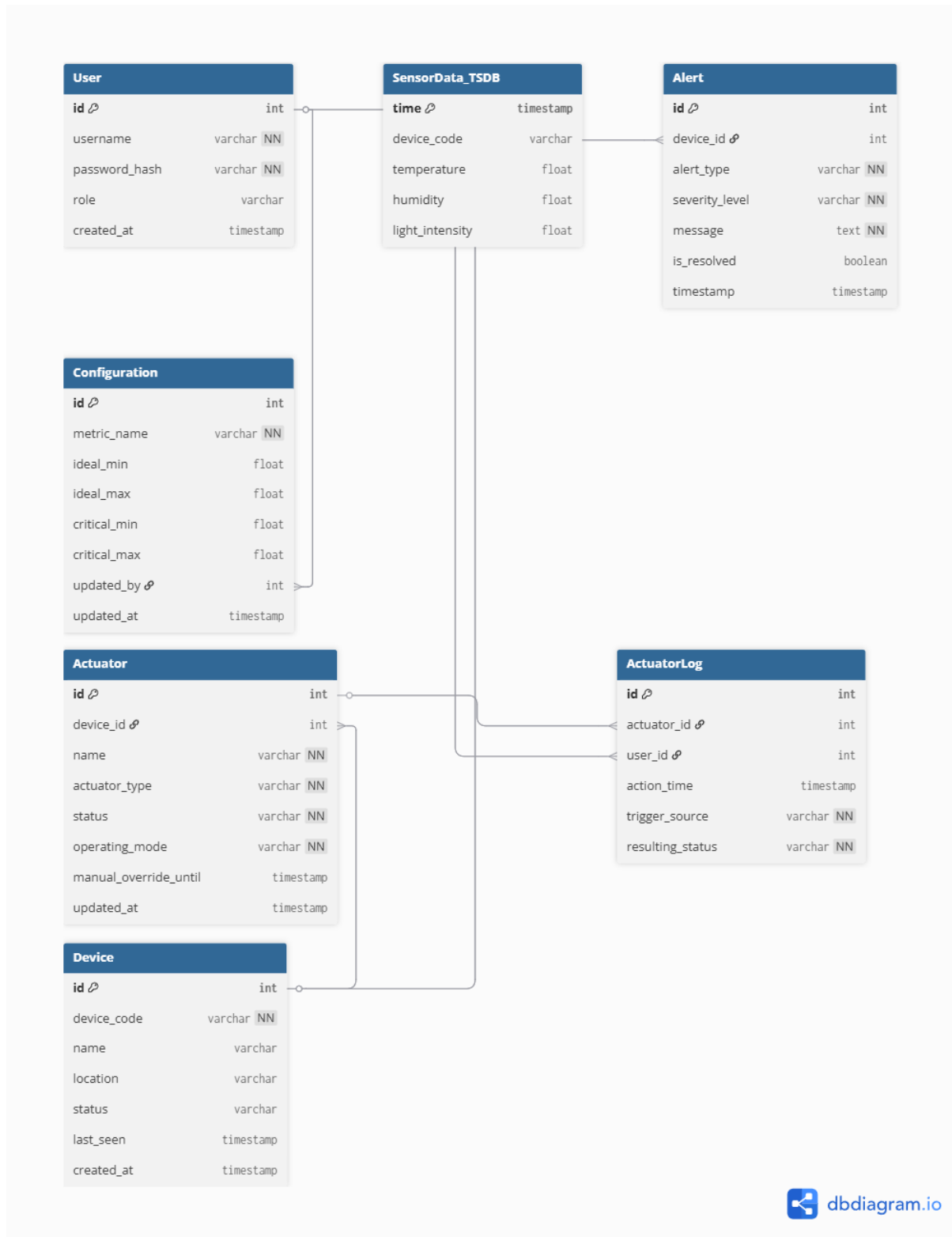


Figure 3: Entity-Relationship Diagram (Relational Schema) and Time-Series Data Flow

The database architecture is strictly segregated into two functional domains:

5.3.1 Time-Series Database (e.g., InfluxDB) for Telemetry

IoT sensor nodes generate continuous streams of telemetry data. Storing this high-throughput data in a relational table quickly leads to bottlenecks and degrades query performance for historical charts.

Therefore, a TSDB is employed to handle environmental monitoring:

- **SensorData** (Measurement): Stores the real-time environmental metrics collected from the edge devices.
 - **Fields:** `temperature`, `humidity`, `light_intensity`.
 - **Tags (Indexed):** `device_id`, `location`.
 - **Timestamp:** The primary index for all records.

This approach facilitates extremely fast write operations and provides built-in downsampling capabilities (e.g., aggregating per-second data into hourly averages) directly at the database level. This significantly reduces the computational load on the backend when rendering data visualizations for the dashboard.

5.3.2 Relational Database (e.g., PostgreSQL/MySQL) for State and Logic

While the TSDB handles continuous telemetry, the RDBMS is responsible for maintaining system state, operational configurations, and user management, where ACID (Atomicity, Consistency, Isolation, Durability) compliance and referential integrity are paramount. The ERD (Figure 3) illustrates these core entities:

- **Actuator:** Represents the physical control devices connected to the system. It stores the `actuator_type`, current physical `status` (ON/OFF), and the current `operating_mode` (AUTO/MANUAL).
- **Configuration:** Stores the user-defined operational thresholds. It includes the `ideal_range` and `critical_range` for each environmental metric. This table ensures the `AutoStrategy` logic has persistent, up-to-date parameters to evaluate against real-time sensor data.
- **ActuatorLog:** Serves as an audit trail. Every time an actuator changes state, a record is created detailing the `action_time`, the `trigger_source` (whether triggered automatically by the system or manually by a user), and the resulting state.
- **User:** Manages authentication and authorization credentials, ensuring secure access to the dashboard and manual control features.
- **Alert:** Records systemic anomalies and critical events, capturing the `alert_type`, `severity_level`, `message`, and the `timestamp`.

By decoupling the high-throughput sensor data from the transactional system data, this hybrid architecture ensures that the relational database remains highly responsive for control logic and user authentication, even when the system scales to support numerous sensor nodes transmitting data simultaneously.

5.4 API and Communication Design

To ensure seamless integration across the distributed architecture—comprising the Edge devices, the centralized Backend, and the Frontend Dashboard—a standardized communication contract is established. The system employs a dual-protocol strategy: MQTT for low-latency, lightweight IoT communication at the edge, and HTTP/REST for client-server web interactions.

5.4.1 MQTT Communication (Edge to Cloud)

The Yolo:Bit edge device communicates with the central system via the Adafruit IO MQTT Broker. To ensure standardized routing and proper data isolation, all MQTT topics strictly adhere to the Adafruit IO naming convention: `<username>/feeds/<feed_name>`. In this structure, `<username>` serves as a placeholder for the designated Adafruit IO account name used by the system.

The data exchange is systematically categorized into two distinct operational flows:

- **Telemetry Publishing (Edge → Cloud):** The edge device periodically samples environmental sensors and publishes the numeric data as string payloads to the cloud.
 - `<username>/feeds/temp-test1`: Transmits real-time temperature data. (Payload example: "32.5")
 - `<username>/feeds/humid-test1`: Transmits real-time air humidity data. (Payload example: "75.0")
 - `<username>/feeds/luminor-test1`: Transmits real-time light intensity data. (Payload example: "850")
- **Command Subscription (Cloud → Edge):** To facilitate remote device management, the edge device continuously subscribes to specific feeds to receive execution commands from the backend's ActuatorService.
 - `<username>/feeds/actuator-fan`: Receives state commands for the ventilation fan. Expected payloads: "ON" or "OFF".
 - `<username>/feeds/actuator-pump`: Receives state commands for the water pump. Expected payloads: "ON" or "OFF".
 - `<username>/feeds/actuator-led`: Receives color state commands for the RGB indicator LED. Expected payloads: "RED" (critical alert) or "GREEN" (stable condition).

5.4.2 RESTful APIs (Frontend to Backend)

The Frontend Dashboard interacts with the Backend via RESTful APIs over HTTP. To support load balancing and reverse proxying, all endpoints are prefixed with `/api/v1/` and routed through the Nginx Load Balancer. The data exchange format is strictly standardized to JSON.

Endpoint	Method	Description & Expected Payload
<code>/auth/login</code>	POST	Authenticates user. Returns a JWT token for authorization in subsequent requests.
<code>/telemetry</code>	GET	Retrieves historical TSDB data. <i>Query Params:</i> <code>?metric=temp&range=7d</code>
<code>/actuators/{id}/toggle</code>	POST	Manually overrides an actuator. <i>Body:</i> <code>{"action": "ON", "duration_min": 5}</code>
<code>/configurations</code>	PUT	Updates ideal and critical threshold ranges. <i>Body:</i> <code>{"metric": "TEMPERATURE", "ideal_max": 28.5, "critical_max": 35.0}</code>
<code>/alerts</code>	GET	Fetches system, security, and environmental alerts. <i>Query Params:</i> <code>?status=unresolved</code>

Table 8: Core RESTful API Endpoints



This API design ensures that the Frontend can independently query historical states and issue manual commands. Simultaneously, the Backend acts as a secure mediator, validating requests and applying the `ManualStrategy` logic before dispatching corresponding MQTT payloads to the edge device via the Adafruit IO broker.

6 Implementation and Integration

6.1 Edge Hardware Setup

6.1.1 Overview of the Role of the Edge Device in the System

- **Primary Functions:** Collect environmental data including temperature, humidity, and light intensity in real-time.
- **Local Processing:** Display information on the Yolo:bit's LCD screen and provide immediate feedback. The RGB LED transitions to red when the temperature is excessively high. The LCD displays a theft alert when the infrared sensor detects a heat source.
- **Network Communication:** Package data and transmit it to the Cloud platform (Adafruit IO) using the MQTT protocol.

6.1.2 Hardware Architecture

- **Central Control Unit:** Based on the ESP32 chip featuring a dual-core CPU with speeds up to 240MHz. Supports Wi-Fi 802.11 b/g/n and Bluetooth connectivity, suitable for flexible IoT applications.

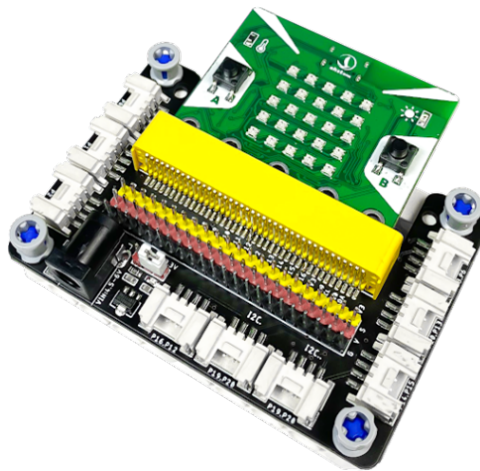


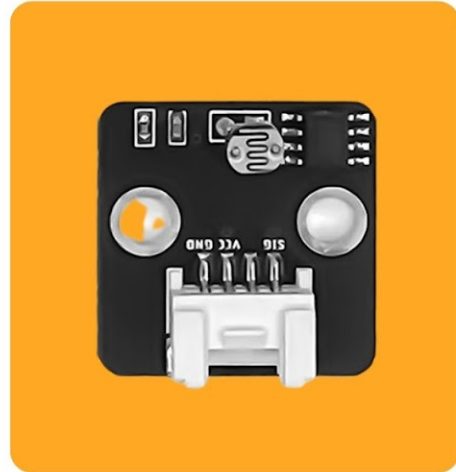
Figure 4: Yolo:Bit Extend Shield Illustration

- **Extension Board:** The Yolo:Bit Extend Shield is a breakthrough expansion board specifically designed for the Yolo:Bit. This board functions to connect the Yolo:Bit pins to the I/O, I2C, and SPI pins of the expansion circuit.
- **Sensor System and Peripherals:**
 - **Temperature & Humidity Sensor (DHT20):** Utilizes the I2C interface for higher accuracy and stability in agricultural environments.
 - **Light Sensor (LDR):** Operates based on resistance changes relative to light intensity; data is processed via the ESP32's ADC (Analog-to-Digital Converter).
 - **Infrared Sensor:** Used to detect obstacles or presence within the cultivation area.
 - **LCD 16x2/OLED Screen:** Connected via the I2C standard to conserve pins, displaying system status and local environmental parameters.

- **RGB LED:** Utilized as a status indicator light. It displays green when the circuit is operating stably and red when the temperature is too high.
- **Servo Motor and Fan System:** Controlled remotely from the Cloud.
- **Power Supply:** Uses batteries or an adapter to ensure stability for the sensor node.



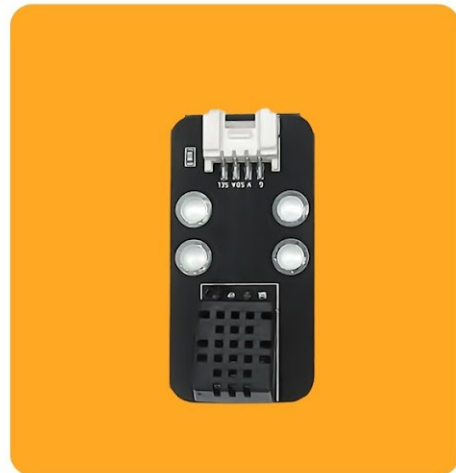
RGB LED



Light Sensor



Light Sensor



Temperature & Humidity Sensor

Figure 5: Peripheral sensors and indicator modules

6.1.3 Connection Process for Data Transmission from Edge to Cloud

1. Setup Phase:

- Configure I/O for the sensors.
- Initialize the Protocol Stack: Connect to Wi-Fi and authenticate the MQTT Client with the Adafruit User/Key.

2. Main Loop:

- **Sampling:** Read data from the DHT20 and LDR every 5 seconds.
- **Local Logic:** If *Temperature* > 35°C, display a warning on the LCD and activate the RGB LED.
- **Publishing:** Send data to the corresponding feeds.

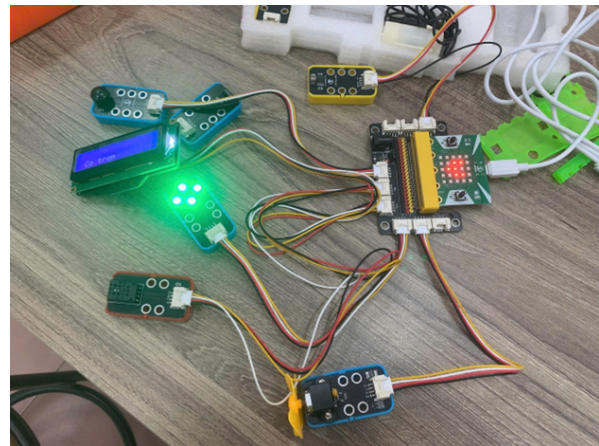
6.1.4 Cloud Connection Configuration (Adafruit IO Integration)

- **Transmission Protocol:** MQTT is lighter than HTTP due to its very small header, reducing energy consumption for the Edge device.
- **Data Structure:** Consists of 3 channels (Feeds) for reporting information:
 - Humid-test1: Channel containing humidity information.
 - Luminor-test1: Channel containing light intensity information.
 - Temp-test1: Channel containing temperature information.

6.1.5 Experimentation and Evaluation



Adafruit IO Dashboard Monitoring



Physical Wiring of the Smart Agriculture System

Figure 6: Experimental results

6.2 Load Balancer Configuration

The Load Balancer is implemented as a custom Node.js middleware that acts as the central traffic controller for the system. To ensure flexibility, maintainability, and seamless deployment across different environments (such as Docker containers), the Load Balancer's operational behavior is entirely driven by a centralized configuration file named `upstream.conf`.

Example `upstream.conf`:

```
{
  "http_port": 8080,
  "backends": [
    { "host": "127.0.0.1", "port": 3001 },
    { "host": "127.0.0.1", "port": 3002 }
  ],
  "adafruit_io": {
    "host": "mqtt://io.adafruit.com",
    "username": "YOUR_ADAFRUIT_USERNAME",
    "key": "YOUR_ADAFRUIT_KEY",
    "topics": [
      "YOUR_ADAFRUIT_USERNAME/feeds/sensor.temperature",
      "YOUR_ADAFRUIT_USERNAME/feeds/sensor.humidity"
    ]
  }
}
```

Structured in JSON format, the `upstream.conf` file strictly decouples infrastructure parameters from the application's source code. This design allows system administrators to scale or modify the network topology without altering the codebase. The file is divided into three key configuration blocks:

- **General Settings (`http_port`):** Defines the primary listening port (e.g., 8080) where the Load Balancer's HTTP Router will intercept incoming traffic.
- **Backend Cluster Definition (`backends`):** Contains an array of active backend server instances, specifying their respective host IP addresses and ports. This array serves as the dynamic routing table for the load distribution algorithm.
- **Middleware Credentials (`adafruit_io`):** Securely stores the connection parameters for the Adafruit IO broker. It includes the `host` URI, the `username`, the active `key` for MQTT authentication, and an array of specific `topics` (telemetry feeds) to monitor.

Based on the parameters defined in this configuration file, the Load Balancer initializes its two core sub-modules:

- **MQTT Subscriber Initialization:** Upon startup, this module parses the `adafruit_io` block to establish a persistent, authenticated MQTT connection. It dynamically subscribes to all listed environmental topics. When telemetry data is published by the Edge devices, the subscriber captures the payload, encapsulates it into a standardized JSON object, and forwards it to the HTTP Router via an internal POST request.
- **HTTP Router and Round-Robin Execution:** The HTTP Router reads the `backends` array to map out available processing nodes. Utilizing a **Round-Robin** scheduling strategy, it sequentially cycles through the list of backend servers for each incoming HTTP request—whether the request originates externally from the Web/App clients or internally from the MQTT Subscriber. This ensures an even distribution of the computational load, mitigates the risk of single-point failures, and maintains high availability across the entire ecosystem.

6.3 Backend API Development and Web Interface Integration

6.4 Data Analysis Module

The Data Analysis Module is responsible for processing, analyzing, and visualizing IoT sensor data streams, including temperature, humidity, and light intensity. This module directly supports key func-

tional requirements such as real-time monitoring, historical data analysis, alert generation, and decision support.

To ensure modularity and scalability, the module is designed as a data processing pipeline consisting of the following components:

- **Data Ingestion and Refresh Mechanism:**

The system retrieves sensor data from cloud data sources. To maintain data freshness and responsiveness, a dual-refresh strategy is applied:

- *Manual refresh*: Users can request the latest data on demand.
- *Automatic polling*: The system periodically updates data every five minutes.

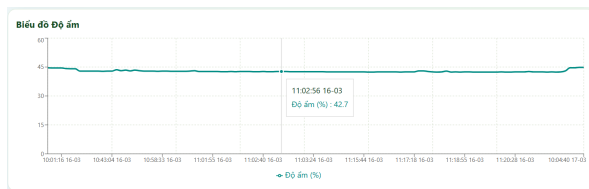
This mechanism ensures consistency between real-time monitoring and historical views while minimizing stale data.



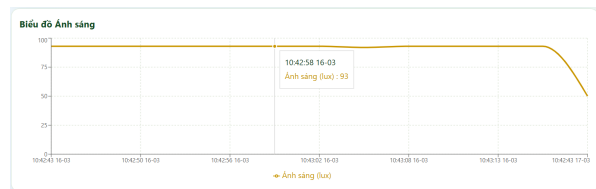
Figure 7: Data Refresh Mechanism

- **Data Visualization:**

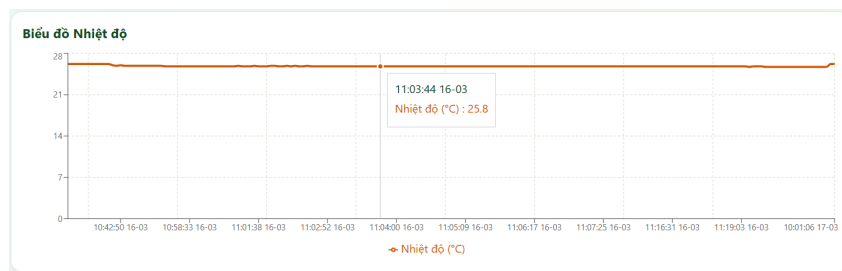
Sensor data is visualized through interactive charts, with separate views for each environmental metric (temperature, humidity, and light intensity). The visualization layer supports dynamic updates based on selected filters and aggregation levels, enabling users to seamlessly switch between detailed and summarized perspectives.



Humidity Dashboard



Light Dashboard



Temperature Dashboard

Figure 8: Dashboard Overview

- **Time Filtering and Data Selection:**

The module provides flexible time-based filtering, including predefined intervals (Day, Week, Month) and custom date range selection. This allows users to analyze specific time periods, such as abnormal environmental conditions or crop growth phases. All filters are applied consistently across metrics to ensure meaningful comparison.



Figure 9: Time Filtering and Date Range Selection

- **Data Aggregation and Trend Analysis:**

To enhance interpretability, the system adapts data granularity based on the selected time scale:

- Raw data is used for short-term, high-resolution monitoring.
- Aggregated data (e.g., average per day, week, or month) is used for long-term analysis.

This approach reduces noise and highlights long-term trends, supporting better decision-making.

- **Status Evaluation and Alert Integration:**

Incoming data is continuously evaluated against predefined *ideal range* and *critical range* thresholds. Each data point is classified into one of three states: Normal, Warning, Critical.

These states are tightly integrated with the system's alert mechanism, triggering visual indicators and notifications on the dashboard, as defined in the functional requirements.

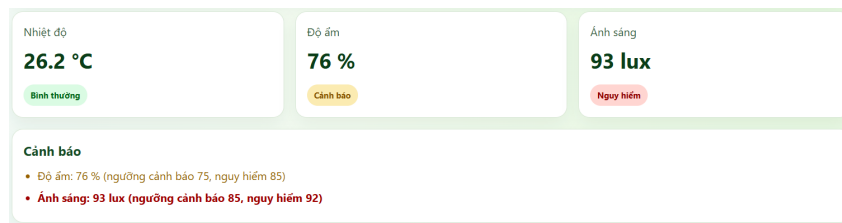


Figure 10: Status Evaluation and Alert

- **Recommendation Engine:**

Based on evaluated conditions and detected trends, the module provides actionable recommendations (e.g., adjusting irrigation, ventilation, or lighting). The current implementation is primarily rule-based, ensuring transparency and reliability. The architecture also allows future integration with AI/ML models or LLM-based services for more adaptive and context-aware recommendations.

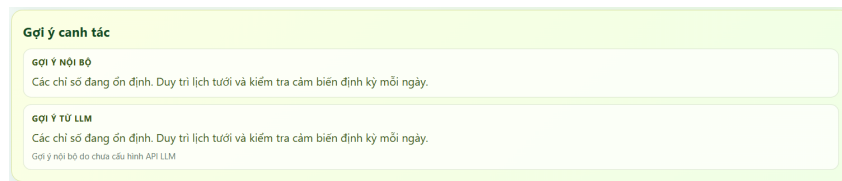


Figure 11: Recommendation Interface

- **Data Export and Persistence:**

The module supports exporting environmental data in CSV format for offline analysis and reporting. Additionally, historical data is retained to enable long-term trend analysis and system optimization.



7 Conclusion and Future Work

7.1 Evaluation of Achieved Results

7.2 Future Development and System Expansion